

Design and Implementation of a Single-Cycle RISC-V Processor (RV32I)

1. Abstract

This report presents the design and implementation of a single-cycle RISC-V processor supporting basic RV32I instructions. The processor was developed using Verilog HDL and simulated using ModelSim, Quartus Prime, and Vivado. The design includes instruction memory, register file, ALU, control unit, data memory, and immediate extender.

2. Objectives

- Design a single-cycle RISC-V processor.
- Implement basic RV32I instructions (ADDI, LW, SW, OR, BEQ).
- Verify functionality through simulation.
- Analyze RTL diagrams and resource utilization.

3. Instruction Set Implemented

ADDI, LW, SW, OR, and BEQ were implemented and tested using an assembly program. Figure 1 represents the RISC-V instructions for which the datapath is created.

PC	Machine Code	Basic Code	Original Code
0x0	0x00900493	addi x9 x0 9	addi x9, x0, 9
0x4	0x00500293	addi x5 x0 5	addi x5, x0, 5
0x8	0xFE54AE23	sw x5 -4(x9)	sw x5, -4(x9)
0xc	0xFFC4A303	lw x6 -4(x9)	L1: lw x6, -4 (x9)
0x10	0x0064A423	sw x6 8(x9)	sw x6, 8 (x9)
0x14	0x0062E233	or x4 x5 x6	or x4, x5, x6
0x18	0xFE420AE3	beq x4 x4 -12	beq x4, x4, L1

Figure 1: RISC-V Assembly instructions on Venus Simulator

4. Instruction Encoding

Instructions were encoded into hexadecimal and stored in little-endian byte format in instruction memory. Instructions and their encoded versions are given below:

4.1 Instructions

```
addi x9, x0, 9           // ADD
    addi x5, x0, 5       // ADD
    sw   x5, -4(x9)      // ADD
L1: lw x6, -4 (x9)        // ADD
    sw x6, 8 (x9)        // ADD
    or  x4, x5, x6       // OR
    beq x4, x4, L1       // SUB
```

4.2 Instructions in HEXADECIMAL:

```
00900493
00500293
FE54AE23
FFC4A303
0064A423
0062E233
FE420AE3
```

4.3 Instructions in Little-Endian

```
93 04 90 00
93 02 50 00
23 AE 54 FE
03 A3 C4 FF
23 A4 64 00
33 E2 62 00
E3 0A 42 FE
```

4.4 Instruction Memory Size

Since a single instruction in RV32i is 32 bits (4 Bytes), this means 7 instructions will consume 28 Bytes.

$$\text{Memory Size} = 7 \text{ instructions} \times 4 \text{ Bytes} = 28 \text{ Bytes} = 28 \times 8 = 224 \text{ Bits}$$

5. Processor Architecture

The processor follows a single-cycle datapath architecture consisting of:

- Program Counter (PC)
- Instruction Memory
- Control Unit
- Register File
- Immediate Extender
- ALU
- Data Memory
- Multiplexers and Adders

6. Control Unit Design

The control unit generates control signals such as ALUSrc, ResultSrc, RegWrite, MemWrite, Branch, and ALUOp based on the opcode field of the instruction.

6.1 Main Decoder Truth Table

Instructions	opcode	alusrc	resultsrc	immsrc	regwrite	memwrite	branch	aluop
addi	0010011	1	0	00	1	0	0	00
lw	0000011	1	1	00	1	0	0	00
sw	0100011	1	x	01	0	1	0	00
and	0110011	0	0	xx	1	0	0	01
beq	1100011	1	0	10	0	0	1	10

6.2 ALU Decoder Truth Table:

Instructions	aluop	func3	func7	alucontrol
addi	00	000	-	000
lw	00	010	-	000
sw	00	010	-	000
or	01	110	0000000	001
beq	10	000	-	010

6.3 Control Unit ModelSim Simulation Waveform

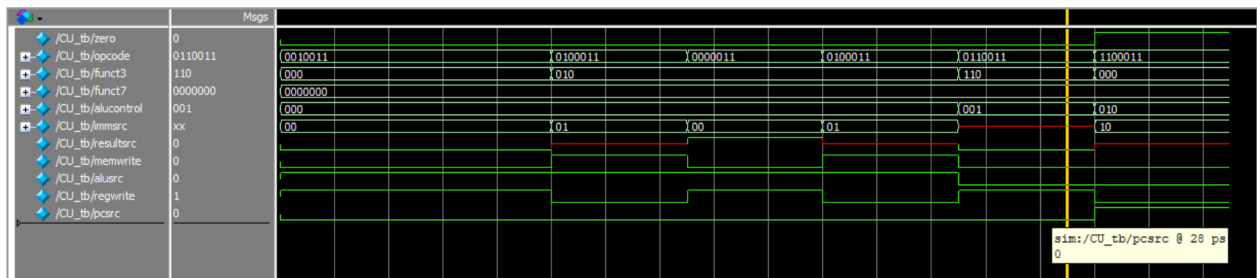


Figure 2: Control Unit Waveform

6.4 Control Unit RTL Diagram

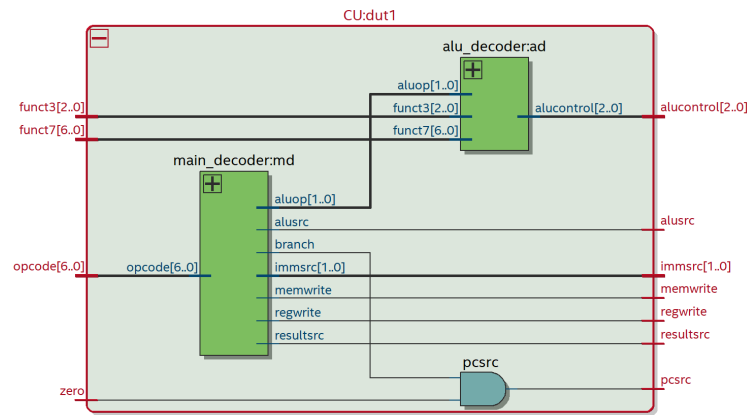


Figure 3: Control Unit

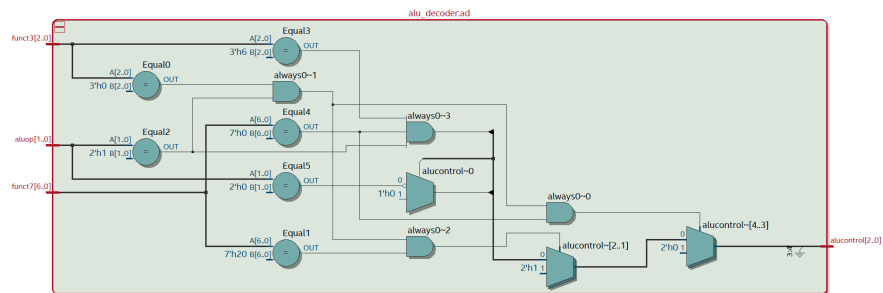


Figure 4: ALU Decoder

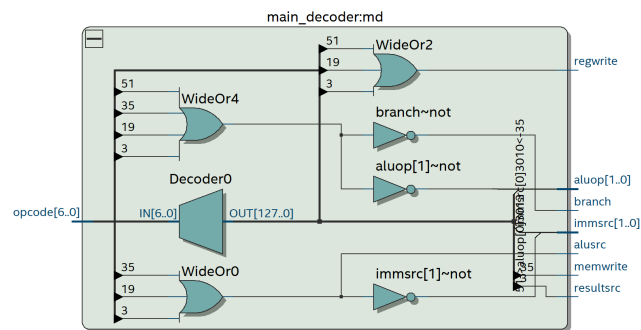


Figure 5: Main Decoder

7. ALU Design

The ALU performs addition, subtraction, and logical OR operations, which are determined by the ALU control signal. When the control value is 0, the ALU performs addition. When the control value is 1, it performs a logical OR operation. When the control value is 2, the ALU performs subtraction.

7.1 ALU ModelSim Simulation Waveform

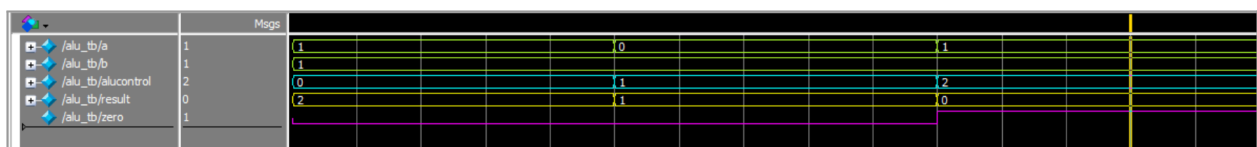


Figure 6: Control Unit

7.2 ALU RTL Diagram

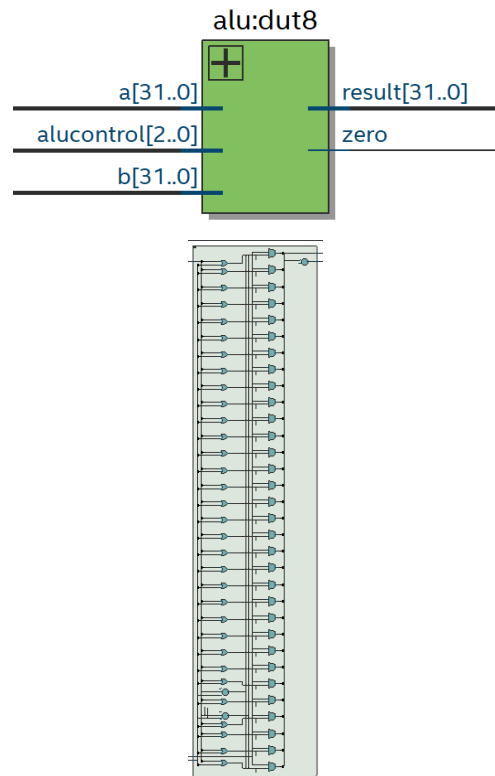


Figure 7: ALU

8. Instruction Memory

Instruction memory in an RV32I processor is a read-only memory (ROM) that stores the program instructions to be executed by the CPU. Each instruction is stored at a unique address and fetched sequentially or through branching based on the program counter. Since it is read-only, the processor cannot modify the instructions at runtime, ensuring that the program code remains fixed while execution proceeds. Instruction memory is essential for providing the processor with the sequence of operations that implement the desired functionality.

7.1 Instruction Memory ModelSim Simulation Waveform

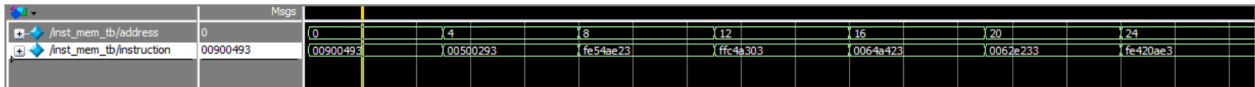


Figure 8: Instruction Memory Waveform

8.2 Instruction Memory Diagram

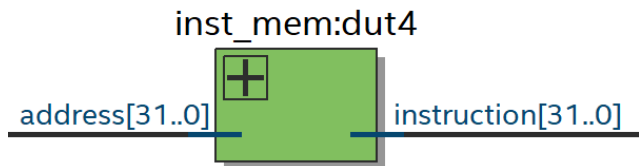


Figure 9: Instruction Memory RTL

9. Data Memory

Data memory is a storage area in a processor where data values are temporarily kept during program execution. It allows the CPU to read and write data needed for computations, using instructions like load (lw) and store (sw). Each memory location has a unique address, and the CPU accesses these locations using effective addresses calculated from base registers and offsets. Data memory is essential for storing variables, intermediate results, and data structures, enabling the processor to perform operations efficiently.

9.1 Data Memory ModelSim Simulation Waveform

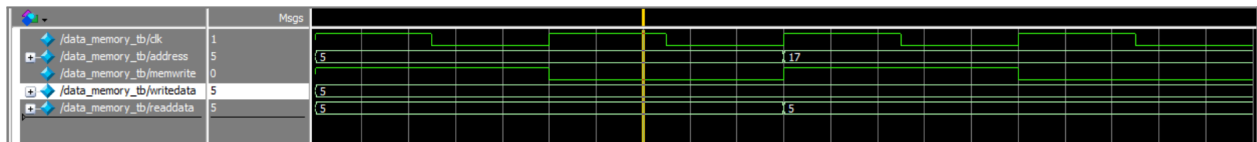


Figure 10: Data Memory Waveform

The instruction `sw x5,-4(x9)` calculated the effective address as 5 and stored the value of `x5` (5) to memory with `MemWrite` enabled. The instruction `lw x6,-4(x9)` accessed memory at address 5, read the value 5, and loaded it into `x6` with `MemWrite` disabled. The instruction `sw x6,8(x9)` stored the value of `x6` (5) at address 17, demonstrating correct data transfer to a new memory location. The instruction `lw x7,8(x9)` read the value 5 from address 17 and loaded it into `x7`, confirming proper read-after-write behavior.

9.2 Data Memory RTL Diagram

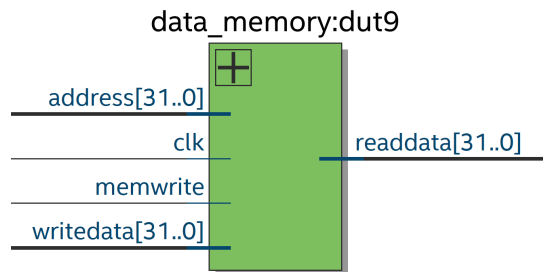


Figure 11: Instruction Memory RTL

10. Register File

In RV32I, the register file is a set of 32 general-purpose registers (x0–x31) used to hold operands and results. It supports reading two source registers and writing to one destination register per instruction, with x0 always fixed at 0.

10.1 Register File ModelSim Simulation Waveform

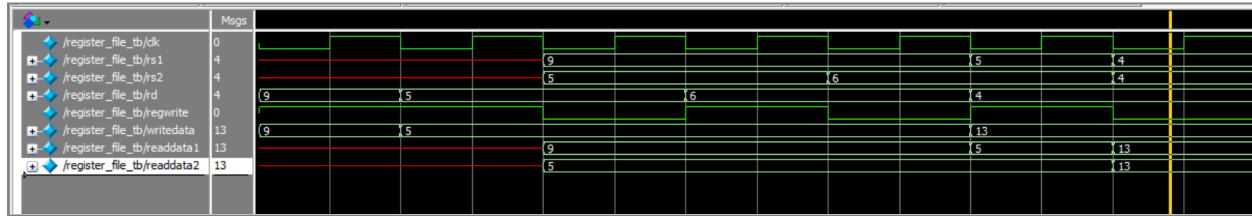


Figure 12: Register File Waveform

The simulation begins with `addi x9, x0, 9` and `addi x5, x0, 5`, which write the immediate values 9 and 5 into registers `x9` and `x5`, respectively, with `RegWrite` enabled and `x0` providing a zero base. The instruction `sw x5, -4(x9)` does not write to the register file, but reads `x9` and `x5` as source operands to calculate the memory address and write data. The `lw x6, -4(x9)` loads the value 5 from memory into `x6`, enabling `RegWrite` and reading `x9` as the base address. The next store, `sw x6, 8(x9)`, again uses `x9` and `x6` as sources without writing to a register. The instruction `or x4, x5, x6` performs a bitwise OR of `x5` and `x6`, writing the result 5 into `x4` with `RegWrite` active. Finally, `beq x4, x4, L1` reads `x4` twice as sources to compare for branching, with no register write. Overall, the waveform shows correct propagation of read and write signals, proper source and destination register usage, and accurate data transfer between registers and memory.

9.2 Register File RTL Diagram

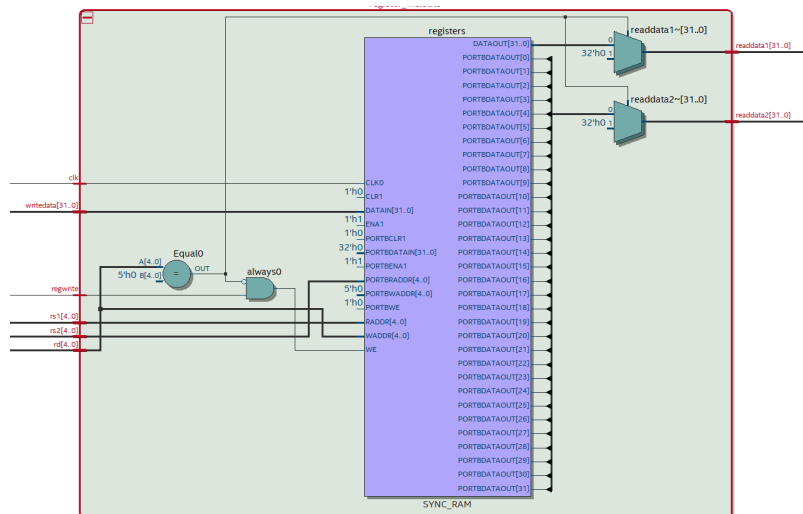


Figure 13: Register File RTL

11. Immediate Extender

The immediate extender is a hardware unit that extracts and sign-extends the immediate field from an instruction to a 32-bit value. It handles I-type, S-type, and B-type instructions, allowing arithmetic, memory, and branch operations to use correct immediate values during execution.

11. 1 Immediate Extender ModelSim Simulation Waveform

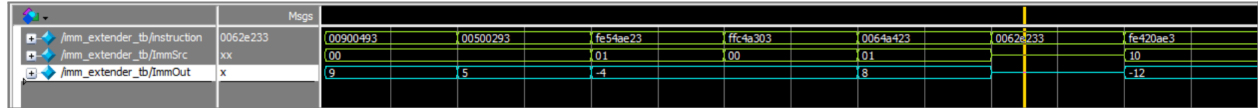


Figure 14: Immediate Extender Waveform

The immediate extender generates the full 32-bit immediate value from instruction fields based on the instruction type. For **ADDI x9, x0, 9** and **ADDI x5, x0, 5**, it sign-extends the 12-bit I-type immediates +9 and +5. For **SW x5, -4(x9)** and **SW x6, 8(x9)**, it forms the 12-bit S-type immediate (-4 and +8) from split instruction fields. The **LW x6, -4(x9)** uses a 12-bit I-type immediate -4. For **BEQ x4, x4, L1**, it constructs a 13-bit B-type immediate -12 for branch calculation. R-type instructions like **OR x4, x5, x6** have no immediate, so the extender is inactive. The waveform shows correct sign extension and output of immediate values for all instruction types.

11.2 Immediate Extender RTL Diagram

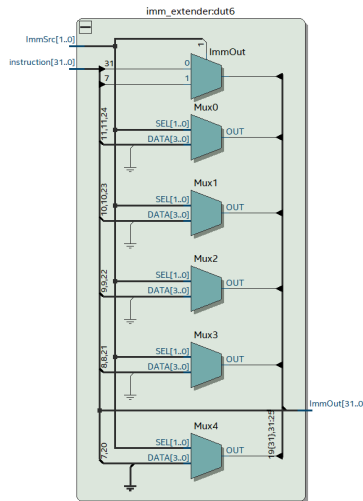


Figure 15: Immediate Extender RTL

12. Program Counter

In RV32I, the Program Counter (PC) is a 32-bit register that holds the address of the next instruction to be executed. It increments by 4 after each instruction fetch for sequential execution, or it can be updated with branch or jump targets for control flow changes. The PC ensures the processor fetches instructions in the correct sequence and is essential for branching, jumping, and loop execution.

12. 1 PC ModelSim Simulation Waveform

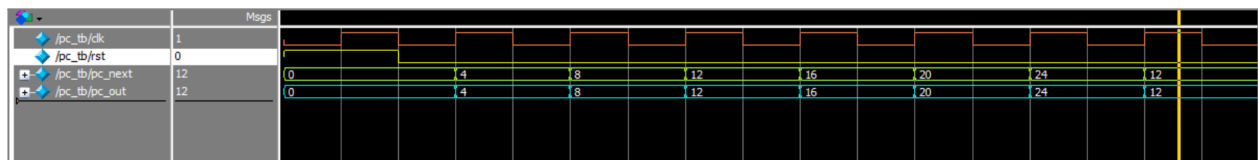


Figure 16: PC RTL

12.2 PC RTL Diagram

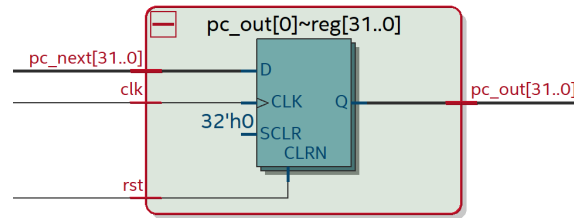


Figure 17: PC RTL

13.Simulation Tools

- ModelSim – Functional verification
- Quartus Prime – RTL synthesis and simulation
- Vivado – Additional synthesis verification

14.Complete RV32i Testbench Results

Simulation waveforms verified the correct execution of instructions, including register updates, memory access, and branching.

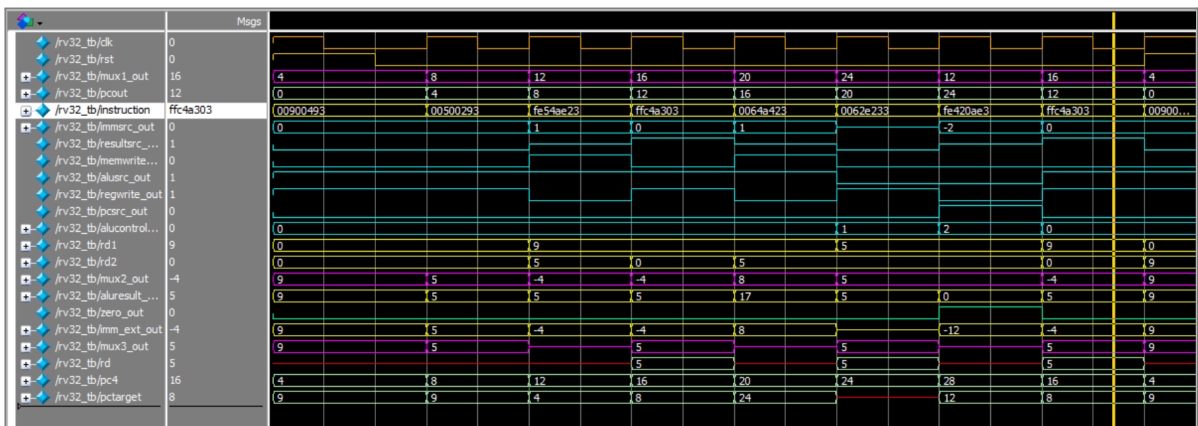


Figure 18: RV32i Waveform

15.RTL Analysis

RTL diagrams confirmed correct datapath connections and module integration.

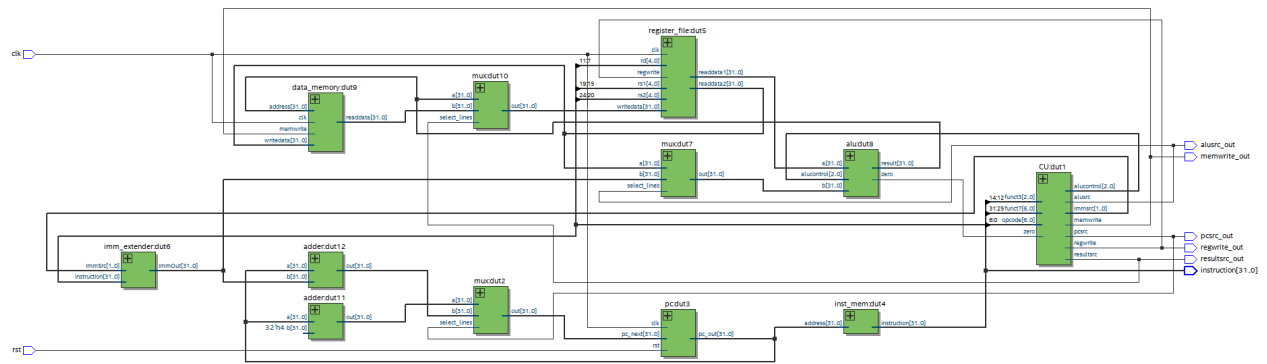


Figure 19: RV32i RTL

16.Synthesis and Implementation using Vivado

Vivado Device View offers a powerful visualization of the design's physical implementation on the FPGA fabric. Therefore, we also synthesized the design using Vivado.

16.1 Synthesis Report

Synthesis reports show efficient use of logic elements and memory resources.

Name	Slice LUTs (63400)	Slice Registers (126800)	F7 Muxes (31700)	F8 Muxes (15850)	Slice (1585 0)	LUT as Logic (63400)	LUT as Memory (19000)	LUT Flip Flop Pairs (63400)	Bonded IOB (210)	BUFGCTRL (32)
rv32i	821	261	144	62	251	773	48	112	7	1
dut3 (pc)	485	5	16	0	156	485	0	5	0	0
dut5 (register_file)	80	0	0	0	37	32	48	0	0	0
dut8 (alu)	0	0	0	0	8	0	0	0	0	0
dut9 (data_memory)	256	256	128	62	182	256	0	0	0	0
dut12 (adder)	0	0	0	0	2	0	0	0	0	0

Figure 20: Synthesis Report

16.2 Utilization Summary Graph

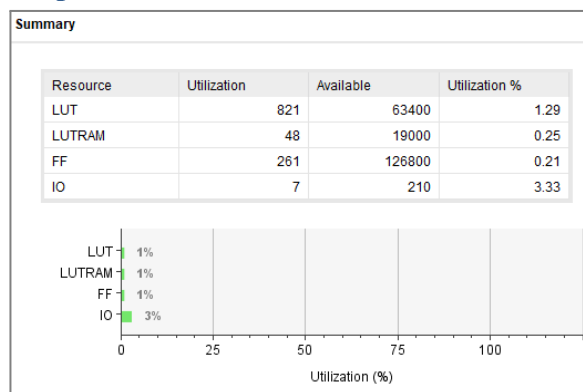


Figure 21: Utilization Summary

16.3 Implemented Design on FPGA

The highlighted teal clusters in the zoomed view represent the specific mapping of the RV32i SCP onto the chip's internal resources. As seen in the left pane, the design is localized within a specific clock region (X0Y1),

which is an indicator of efficient Placement and Routing (P&R). By clustering the logic together rather than scattering it across the die, Vivado minimizes the physical distance signals must travel. This reduced wire length is critical for achieving high-speed timing closure, as it lowers interconnect delay and parasitic capacitance. Furthermore, the visualization reveals that while the design is computationally dense, it utilizes only a small fraction of the total available FPGA area (X0Y0 through X1Y3). This low Resource Utilization profile demonstrates that the architecture is highly optimized for hardware efficiency, leaving substantial "white space" on the device for scaling up the input depth or integrating the processor into a larger hardware accelerator system.

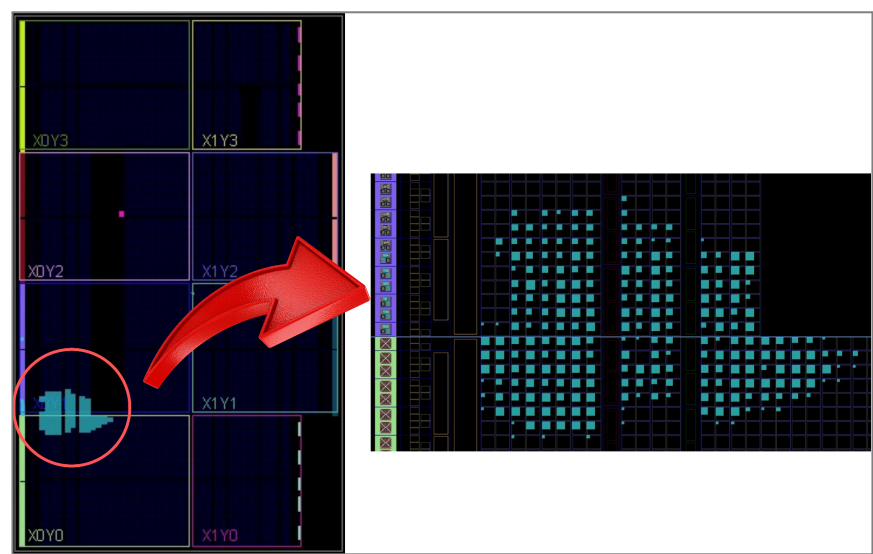


Figure 22: Utilization Summary

16.4 Simulation Waveform and RTL diagram

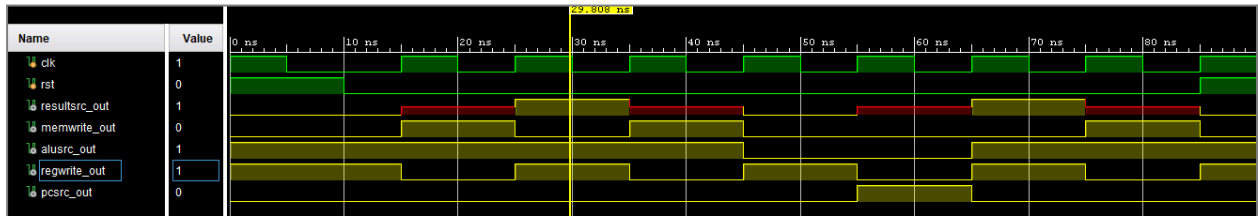


Figure 23: Vivado Simulation Waveform

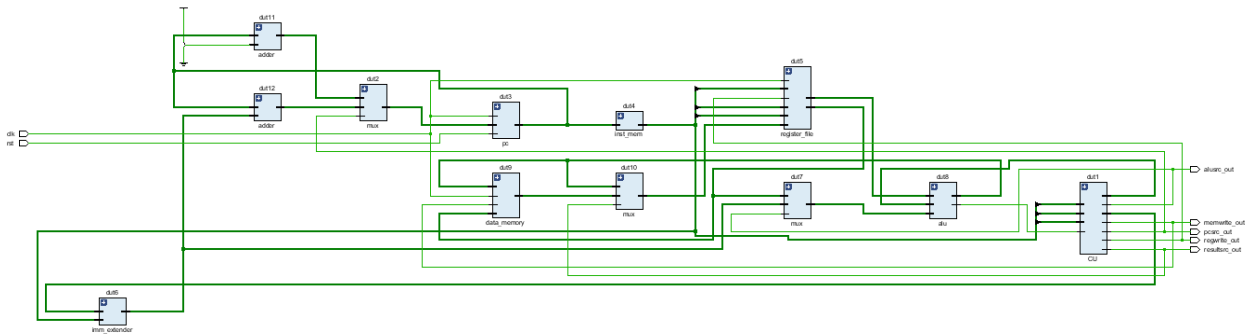


Figure 24: RTL Schematic

17. Conclusion

The single-cycle RV32I processor was successfully implemented and verified. All targeted instructions executed correctly, validating the datapath and control logic.